

MultiUserPaint - Kod Analizi

Tüm Class'ları Açıklayan Teknik Dokümantasyon

Oluşturma Tarihi: 13.06.2026 21:37

RabbitMQ Tercihinin Nedenleri ve Avantajları

1. RabbitMQ Nedir ve Neden Kullanıldı?

Arkadaşlar, bu projemizde biz mesaj kuyruk sistemi olarak RabbitMQ kullanmayı tercih ettik. RabbitMQ, AMQP protokolüne dayalı açık kaynak bir mesaj aracısı (message broker). Basitçe söylemek gerekirse, istemciler ile sunucu arasında veri gönderirken, doğrudan bağlantı kurmak yerine, bir aracı aracılığıyla iletişim kuruyor. Bu aracı, mesajları saklıyor, yönetiyor ve doğru yerlere gönderiyor.

2. RabbitMQ'nun Temel Avantajları

A) Asenkron İletişim (Asynchronous Communication)

RabbitMQ ile sunucu mesajları hemen göndermek zorunda değil. Mesaj kuyruğa konuyor ve alıcı hazır olduğunda alıyor. Bu sayede sunucu istemciyi beklemek zorunda kalmaz. Tek yönlü haberleşme gibi düşünebilirsiniz: birisi mektup yıkıyor, öteki hazır olduğunda okuyor.

B) Fanout Exchange (Yayın-Abone Modeli)

RabbitMQ'de fanout exchange kullanarak bir mesajı birden çok alıcıya aynı anda gönderebiliyoruz. Örneğin: bir kullanıcı sisteme giriş yaptığında, o andan itibaren sistemde olan HERKES bunu öğreniyor. TCP'de bunu yapmak için sunucu tüm istemcileri tek tek kontrol edip mesaj göndermek zorunda. RabbitMQ'de ise exchange mesajı otomatik olarak tüm abone olan kuyruklara tutuyor.

C) Dosya-Spesifik Broadcasting (paint.file.{fileId})

Bir dosya üzerinde çalışan istemcilere sadece o dosyayla ilgili değişiklikleri gönderebiliyoruz. Diyelim 50 tane dosya açık var. Dosya 3 değiştiğinde, sadece dosya 3'ü açan istemciler haberdar oluyor. Diğer 49 dosyayı açan istemciler hiçbir şey almıyor. Bu çok daha verimli bir sistemdir.

D) Bağlantı Bağımsızlığı (Connection Independence)

TCP ile sunucu ve istemci arasında direkt bir bağlantı olması gerekiyor. İstemci offline olursa mesaj kayboluyor. RabbitMQ'de ise mesajlar kuyruklarda bekliyebiliyor. İstemci geri geldiğinde kuyruklanan tüm mesajları alabilir. Ayrıca sunucu ve istemci arasında firewall olsa bile, ikisi de RabbitMQ aracısına bağlanabilirse iletişim kurabiliyor.

E) Ölçeklenebilirlik (Scalability)

TCP ile her yeni istemci için sunucuda yeni bir thread veya bağlantı açması gerekiyor. Binlerce istemci olduğunda sunucu çok yüklenebilir. RabbitMQ ise bu yükü kendi üzerine alıyor. Sunucu sadece RabbitMQ'ya bağlanıyor, istemciler de bağlanıyor. RabbitMQ merkezi yöneticisi rolünü oynuyor.

F) Garantili Teslimat (Delivery Guarantee)

RabbitMQ, mesajın alıcıya ulaştığını teyit ediyor. TCP'de birisi mesajı kaçırsa yeniden gönderme işini programcının yazması gerekiyor. RabbitMQ bunu otomatik yapar. Mesaj alındıktan sonra onay (ACK) gelirse RabbitMQ mesajı siliyor, yoksa tekrar gönderebiliyor.

3. TCP vs RabbitMQ Karşılaştırması

Özellik	TCP (Direkt Socket)	RabbitMQ (AMQP)
Bağlantı Modeli	Direkt sunucu-istemci	Merkezi aracı (Broker)
İletişim Türü	Senkron (blocking)	Asenkron (non-blocking)
Broadcast	Manuel olarak tüm istemcilere gönder	Fanout exchange otomatik
Bağlantı Kesintisi	Mesaj kaybı	Kuyrukta bekler, sonra teslim
Sunucu Yüğü	Her istemci için resource	Merkezi yönetim, az yük
Ölçeklenebilirlik	Sınırlı (thread havuzu)	Yüksek (binlerce istemci)
Firewall/NAT	Sorun olabilir	Her iki taraf aracıya bağlanırsa çalışır
Teslimat Garantisi	Yok, timeout ile kontrol	Evet, ACK mekanizması
Seçici Teslimat	Zor (manuel filter)	Kolay (Exchange + Binding)
Yapılandırma	Basit ama sınırlı	Karmaşık ama çok esnek

4. Bu Projede Neden RabbitMQ Tercih Ettik?

Bu MultiUserPaint uygulamasında çok sayıda istemcinin aynı anda paylaşılan dosyalar üzerinde çalışması gerekiyor. Birisi çizim yaparken diğerleri bunu anında görmek zorunda. TCP ile bunu yapmak mümkün ama çok karmaşık. Çünkü her çizim olayında sunucu tüm ilişkili istemcileri bulup ilk mesaj göndermek zorunda. RabbitMQ ile ise: bir dosya opened ise, biz o dosyaya ait bir exchange kuruyoruz. O dosyayı açan tüm istemciler otomatik olarak o exchange'e abone oluyor. Çizim mesajı geldiğinde, RabbitMQ otomatik olarak tüm abone olanlara dağıtıyor. Çok daha temiz ve güvenilir.

5. RabbitMQ Architecture Bu Projede

paint.server - Tüm istemcilerin sunucuya gönderdiği mesajlar için ortak kuyruk

paint.global - Tüm istemcilere ulaşması gereken mesajlar (USER_JOIN, FILE_LIST)

paint.file.{fileId} - Spesifik bir dosyayı açan istemcilere giden mesajlar (DRAW_BROADCAST)

paint.client.{id} - Her istemcinin özel yanıt kuyruğu (point-to-point mesajlar)

Bu yapı sayesinde ađ trafıđı çok azalıyor. Doğru mesaj doğru yerlere gidiyor. Sunucu hiçbir filtreleme yapmak zorunda kalmıyor, RabbitMQ bunu halletmiş. Kodumuz daha temiz, daha bakımlanabilir ve ölçeklenebilir oluyor.

I. Protocol & Ortak Katman (Foundation)

- 1. ProtocolConstants
- 2. MessageType
- 3. FSMState
- 4. Message
- 5. MessageEncoder
- 6. MessageDecoder
- 7. MQConfig

II. Sunucu Katmanı (Server)

- 8. PaintServer
- 9. NIOSelector
- 10. ClientSession
- 11. MessageDispatcher
- 12. MessageFramer
- 13. LoginHandler
- 14. FileHandler
- 15. DrawHandler
- 16. ClipboardHandler
- 17. FileStore
- 18. FileMetadata
- 19. SessionRegistry
- 20. AutoSaveScheduler
- 21. MQBroker
- 22. MQServerTransport

III. İstemci Katmanı (Client)

- 23. PaintClient
- 24. ConnectionManager
- 25. NetworkReader
- 26. NetworkWriter
- 27. MQNetworkReader
- 28. MQNetworkWriter
- 29. MainFrame
- 30. CanvasPanel
- 31. ToolbarPanel
- 32. FileListPanel
- 33. LoginDialog

BÖLÜM I: Protocol & Ortak Katman (Foundation)

Yazılımın temel altyapısını oluşturan sınıflar. İstemci ve sunucu arasında veri iletişimini standardize eden protokol tanımları, mesaj formatları ve durum yönetimi burada yer alır.

1. Protocol Constants

Burada, arkadaşlar, biz bir iletişim protokolünün temel kurallarını tanımlıyoruz. Aynı bir oyunun kuralları gibi düşünebilirsiniz. Herkes aynı kuralı bilirse oyun oynayabilir. Bu sınıf da istemci ve sunucunun "ortak dili" gibi çalışıyor.

Temel Kurallar:

Önce sihirli sayı diyoruz (0xCAFE). Bir mesaj geldi mi, biz ilk bakıyoruz: "Hey, sihirli sayı doğru mu?" Doğruysa, bu ağımızın mesajı demek. Yanlışsa, hata var veya başka bir protokolden veri geldi. Sürüm numarası (VERSION = 0x01) ise gelecekte protokolü güncellediğimiz zaman ne yapacağımızı söylüyor. "Bu mesaj 1 numaralı protokol ile yazıldı" gibi.

Başlık boyutu (HEADER_SIZE = 8 byte) sabit. Bu 8 byte içinde sihirli sayı, sürüm, mesaj türü ve payload uzunluğu yazılı oluyor. Böylece sunucu ilk 8 byte'ı okuyunca hangi veriyi alacağını biliyor.

Maksimum payload (10 MB) sınırı koyuyoruz. Bir resim dosyası (800×600×4 byte = ~2 MB) gelse bile, birisi bir terabaylık dosya göndermeye çalışmadan koruma altına alıyoruz. Sunucunun hafızası sınırsız değil çünkü.

Port numarası, buffer boyutu, kullanıcı adı karakter sınırı, auto-save aralığı, keepalive timeout... Hepsi burada merkezi olarak tutuluyoruz. Eğer biz buradaki 60 sayısını 120 yaparsak, sistem her 120 saniyede dosyaları kaydeder. Başka hiçbir yerde değişiklik yapmamız gerekmez. Bu da kodun bakımını çok kolaylaştırıyor.

2. MessageType

Şimdi şu mesaj türlerini tanımlıyoruz. Sistemi düşünün: kullanıcı giriş yapabilir, dosya oluşturabilir, çizim yapabilir, pano işlemi yapabilir... Her aksiyon farklı bir mesaj türü. Biz tüm bu türleri enum olarak tanımlıyoruz. Her birine benzersiz bir heksadesimal kod veriyoruz.

Mesaj Türleri Kategorileri:

Giriş mesajları (0x01, 0x02, 0x03): Kullanıcı sisteme giriş yapıyor, başarılı mı başarısız mı.

Dosya mesajları (0x10-0x1A): Dosya oluştur, aç, kaydet, sil, listesi iste... Tüm dosya işlemleri.

Çizim mesajları (0x20, 0x21): Kullanıcı fare ile çizim yapıyor, ben bunu sunucuya "Hey, bu dosyada şu noktadan şu noktaya çizgi çiz!" diye mesaj gönderiyorum.

Pano mesajları (0x30-0x33): Copy, Cut, Paste. Çok istemcili ortamda birinin kestiği şey diğerinin bilgisayarında paste edilebiliyor.

Kullanıcı mesajları (0x40-0x43): Birisi sisteme giriş yaptı, çıktı, tüm istemcilere "Hey, şu kişi sistemde" diye haber veriyoruz.

Canvas anlık görüntü (0x60-0x62): Yeni birisi bir dosyayı açtı. Dosya zaten açık olan birisi var mı? Varsa onun canvas'ından şu anın resmi gönder, disk'ten okuma. Bu sayede herkesin aynı görüntüye bakması garantileniyor.

Keepalive (0xF0, 0xF1): Ping-pong. Aradan uzun süre ses çıkmazsa sunucu "Hey, yaşıyor musun?" diye ping yolluyor. İstemci pong ile cevap veriyor. "Evet yaşıyorum, endişelenme" gibi.

Hata mesajı (0xFF): Birisi yanlış bir işlem yaptı, sunucu hata mesajı gönderiyor.

3. FSMState

Sonlu Durum Makinesi diyoruz buna. Tarihiyle basit: bir oyuncu düşünün. Oyunun başında bekleme durumunda. Menu açılıyor, oyunu seçiyor, oyuna giriyor. Oyun sırasında birisi oyunu durduruyor, tekrar başlıyor... Bu durumlar FSM durumları. Sunucuda da her bağlanan istemci bu durumlardan birinde oluyor.

Sunucu Tarafında Durumlar:

HANDSHAKE: İstemci bağlandı ama henüz giriş yapmadı. Sunucu "Bana kim olduğunu söyle" diyor. Bu durumda sadece LOGIN_REQ mesajı kabul ediliyor. Başka mesaj gelirse, sunucu "Önce giriş yap!" diye hata veriyor.

ACTIVE: Giriş başarılı. Şimdi kullanıcı dosya açabilir, çizim yapabilir, pano işlemi yapabilir. Tüm normal operasyonlar bu durumda gerçekleşiyor.

SAVING: Dosya diske yazılıyor. Sunucu bu sırada yoğun. Mesajlar gelse bile, ping-pong dışında işlenmiyor. Disk yazması bittikten sonra tekrar ACTIVE oluyor.

CLOSING: Güzelce çıkıyor. Tüm kaynakları temizliyor, dosyaları kapatıyor. Sorun yok, sana da hoşça kal diyecek.

ERROR: Aaaa bir hata oldu! Düzeltilemiyor. Bu oturum artık işe yaramıyor, kapatılacak. İstemci yeniden bağlanması gerekiyor.

İstemci Tarafında Durumlar:

DISCONNECTED: Sunucuya bağlı değil. İstemci uygulaması açık olsa da, hiçbir verisi sunucuda yok.

CONNECTING: "Hey sunucu, bana açık mı?" Socket açıyor, bağlantı kuruyor.

LOGGING_IN: "Benim adım şu, al giriş yap beni" LOGIN_REQ göndermiş, cevap bekliyor.

CONNECTED: "Tamamdır, artık ben de senin gibi aktif!" Giriş başarılı, dosyaları görüyor, çizim yapabiliyor.

RECONNECTING: İstemci düştü ama sunucu henüz kapamadı. Tekrar bağlanmaya çalışıyor. Bazen bağlantı kesintileri olur (internet problemi), otomatik yeniden bağlanılıyor.

4. Message

Bu çok basit bir sınıf. Bir mesajın en temel şeklidir. İçinde sadece iki şey var: mesajın türü (MessageType) ve taşıdığı veriler (byte dizisi). Tamamen değişmez (immutable), bir kere oluşturulduktan sonra değiştirilmiyor. Güvenli ve basit.

Kullanımı çok basit: `Message msg = Message.of(MessageType.LOGIN_OK, payload);` Ve o kadar. Artık bu message istediğimiz yere gönderilebiliyor.

5. MessageEncoder

Bu sınıf yazılı verileri ağ üzerinden gönderilebilecek byte dizilerine dönüştürüyor. Biz programcılar Java nesneleriyle rahat çalışıyoruz (String, int, byte[]). Fakat ağ üzerinden sadece byte gönderilmek zorunda. İşte buraya MessageEncoder devreye giriyor.

Örneğin: Kullanıcı "Ali" isminde giriş yapmak istiyor. Biz bunu şöyle yapıyoruz:

```
byte[] payload = MessageEncoder.encodeLoginReq("Ali");
```

MessageEncoder ne yapıyor? "Ali" stringini UTF-8'e dönüştürüyor, önüne de uzunluğunu (2 byte) yazıyor. Sonra bunu bir Message'a sarıyor, ağ formatını (

MAGIC, VER, TYPE, LENGTH, PAYLOAD) ekliyor ve son olarak ağ üzerinden gönderilebilecek byte dizisini veriyor.

6. MessageDecoder

Bu, MessageEncoder'ın tam tersi. Ağdan gelen byte dizilerini (ki bunlar garip, başında sihirli sayı falan var) tekrar insanın anlayabileceği şekle çeviriyoruz.

Örneğin sunucu MESSAGE geldi, ağdan. MessageDecoder.decodeLoginReq() çağırıyoruz. O da byte dizisini parse ediyor, sihirli sayıyı ve başlığı çıkartıyor, geriye kalan payload'ı alıyor, onu da UTF-8'den string'e çeviriyor. Ve sonuç: LoginReqPayload nesnesi, içinde username = "Ali" yazıyor. Çok rahat!

Decoder içinde LoginReqPayload, LoginOkPayload, FileCreateReqPayload gibi payload türleri var. Bunlar basit POJO sınıfları (Plain Old Java Objects), sadece verileri tutuyorlar. Message türüne göre doğru payload sınıfı kullanılıyor.

7. MQConfig

RabbitMQ kullandığımızda, mesajların nasıl yönetileceğini tanımlıyoruz. Bu konfigürasyondaki "topoloji" demek, kuyruklar ve exchange'ler nasıl bağlanacak?

RabbitMQ Topoloji Detaylı Anlatım:

Şöyle düşünün: Bir postane hayal edin. Postane birden çok kutusu var. Sunucudan gelen tüm mesajlar paint.server kutusuna gidiyor. RabbitMQ bu kutuyu okuyuyor, mesajı alıyor ve şuna şu kutuya koy diyerek dağıtıyor.

paint.global: Tüm istemcilere ulaşması gereken mesajlar. Birisi sisteme giriş yaptı (USER_JOIN), herkes bunu bilmeli. Bunlar paint.global fanout exchange'e gidiyor. Fanout exchange ne yapıyor? "Bunu bana bağlı olan herkese gönder" diyor. Tüm istemcilerin kuyrukları buna bağlı olduğu için, herkes mesajı alıyor.

paint.file.{fileId}: Spesifik bir dosya için exchange. Dosya 5 açılmış olsun. Dosya 5 üzerinde çizim yaparsan, DRAW_BROADCAST paint.file.5 exchange'ine gidiyor. Paint.file.5'e abone olanlar (sadece dosya 5'i açanlar) mesajı alıyor. Diğer dosyaları açanlar hiçbir şey almıyor. Verimli, değil mi?

paint.client.{id}: Her istemcinin özel kutusu. Benim client ID'im 12345. Sunucu bana mesaj göndermek isterse, paint.client.12345'e koyuyor. Sadece ben (client 12345) onun abonesi, sadece ben alıyorum. Point-to-point iletişim bu.

Konfigürasyon dosyası RabbitMQ'nun bağlantı detaylarını da tutuyor: Host (localhost), port (5672), kullanıcı adı (guest), şifre (guest)... Bu da merkezi bir yerde tutulup, başka yerden değişiklik yapılmıyor.

BÖLÜM II: Sunucu Katmanı (Server)

Şimdi arkadaşlar, sunucu tarafına geçiyoruz. Sunucu tüm istemcilerin bağlantılarını yönetiyor, paylaşılan dosyaları diske kaydediyor ve mesajları yönlendiriyor. TCP ile çalışabilir, ya da RabbitMQ ile çalışabilir. İkisi de tamamen işliyor.

8. PaintServer

Bu tüm sunucunun başı. Uygulamayı çalıştırdığında en önce bu sınıf çalışıyor. Java main() metodu burada. Ne yapıyor? Bütün sistemi başlatıyor, kullandığı nesneleri oluşturuyor.

Kullanım:

TCP modunda: `java PaintServer 9090 /path/to/files`

RabbitMQ modunda: `java PaintServer --mq rabbitmq.example.com /path/to/files`

Başlatılan Altyapı:

SessionRegistry: "Sistem, kim bağlı?" sorusunun cevabı. Tüm istemcileri burada tutuyor.

FileStore: Disk'te dosyaları okup yazıyor. $800 \times 600 \times 4 \text{ byte} = 1.92 \text{ MB}$ resim.

DiskPool: 2 thread havuzu. Disk çok yavaş olabilir, bu yüzden bu işleri ayrı threadlerde yapıyoruz. Ana sunucu thread'i bloke olmaz.

MessageDispatcher: Mesajlar geldiğinde, bunlar kimin işine giriyorsa ona yönlendirir. Giriş mesajı → LoginHandler, Çizim mesajı → DrawHandler, vb.

LoginHandler, FileHandler, DrawHandler, ClipboardHandler: Her biri farklı türde mesajları işleyen uzmanlar.

AutoSaveScheduler: Her 60 saniyede bir tüm açık dosyaları diske kaydediyor. Veri kaybı olmasın diye.

NIOSelector (TCP) veya MQServerTransport (RabbitMQ): Ağ iletişimini yönetiyorlar. Hangisini seçtiysen ona göre başlatılıyor.

9. NIOSelector

Bu TCP modundaki kalp atışı. NIO demek Non-Blocking I/O. Bunun neden önemli olduğunu anlatayım. Normal socket programming'de, her istemci için bir thread açarsın. 1000 istemci = 1000 thread. Her thread hafıza tutuyor, context switch çok. Sunucu çöp olur.

NIO ile? Tek bir thread tüm 1000 istemciyi yönetiyor. Nasıl? Selector diye bir şey var. Selector diyor: "Ey tüm socketler, hangisinin veri gelmişse bana söyle." Bir socket "Benim veri var" derse, selector gider o socket'ten okur. Başkası "Benimde var" derse, ondan da okur. Sırayla herkese bakar. Bu çok verimli ve hızlı.

Yapıyor Olduğu İşler:

Accept: Yeni istemci mi geliyor? (Mesela port 9090'a bağlanmaya çalışıyor) Selector kabul ediyor, o istemci için ClientSession oluşturuyor, registry'ye ekliyor.

Read: Hangi istemcilerin veri gelmişse onlardan okuyor. Okunan byte'ları MessageFramer'a besliyor. MessageFramer mesaj birleştiriyor ve "Bir tam mesaj hazır!" dediğinde, Dispatcher'a gönderiyor işlenmek üzere.

Write: İstemciye yanıt yazılacak mı? (writeQueue'de mesaj var mı?) Varsa, session'ın writeQueue'sinden alıp istemciye gönderiyor.

Keepalive: Aradan 90 saniye veri almadığı istemci var mı? Evet, öldürmüş kabul ediyoruz. Bağlantı kapanıyor. Ping-pong mekanizması de var aslında, ama çok uzun sürerse bu çalışıyor.

10. ClientSession

Bağlı her istemci için sunucuda bir ClientSession nesnesi var. Bu obje, "Ben kimim? Hangi dosyaları açtım? Panom'da ne var? Son ping zamanı ne zaman?" gibi tüm detaylarını tutuyor. Bir istemci kaç yaşındaki çocuksa, ClientSession onun "özet bilgisi".

İçerdiği Ana Veriler:

sessionId: Her istemci için benzersiz numara. "Sen 12345 numara mı? O zaman seni tanıyorum."

channel (TCP modunda): SocketChannel. Doğrudan haberleşme tüpü gibi. RabbitMQ'de null.

state: FSMState. Şu anda HANDSHAKE mı, ACTIVE mi, SAVING mi durumda?

username: Giriş yaptığı kullanıcı adı. "Ali" giriş yaptıysa, username = "Ali".

Mesaj çerçeveleme tamponları (readBuffer, headerBuf, payloadBuf): Gelen veriler burada birleştiriliyor. Örneğin, 5 bayt geldiyse burada tutulur, sonra 3 bayt daha gelse, toplam 8 bayt oluyor ve başlık tamam diyelim.

writeQueue: Gönderilecek mesajlar kuyruk. SessionRegistry mesajı buraya koyuyor, NIOSelector'ın write loop'u buradan alıp istemciye gönderiyor.

openFileIds: Şu istemci hangi dosyaları açmış? Set olarak tutuyor. Dosya 1, 3, 5 açıksa, {1, 3, 5} seti tutuyor.

clipboardData: Kopya-yapıştır işlemleri. Kullanıcı Ctrl+C yaptıysa, o bölgenin pixel verisi burada saklanıyor. Ctrl+V yaptığında, buradan çekiliyor.

RabbitMQ modundaki ekstra veriler: mqReplyTo (istemcinin özel kuyruk adı), mqSink (gönderme işlevi).

11. MessageDispatcher

Mesaj yönlendiricisi. Bir mesaj geldiğinde, "Bu neyin işi? Kimin yapması lazım?" diye soruyor. Giriş mesajı geldi mi → LoginHandler'a git. Çizim mesajı geldi mi → DrawHandler'a git. Dosya mesajı geldi mi → FileHandler'a git. Trafik polisi gibi.

Ama bir kural var: istemcinin durumu önemli. HANDSHAKE'de sadece LOGIN_REQ kabul ediliyor. Başka mesaj gelirse, "Hey! Önce kendini tanı!" hata veriyor. ACTIVE'de, tüm normal mesajlar işleniyor. SAVING'de, disk yazması bitmeden mesajlar (ping hariç) işlenmiyor. ERROR'de, kapanış başlatılıyor.

12. MessageFramer

Ağdan gelen ham byte akışını tamamlanmış mesajlara dönüştüren sihircisi. Şöyle bir sorun var: Network sabit paketler göndermiyor. Kümlaştırabilir. Bölebilir. Mesaj 1000 byte'sa, 500 geliyor, sonra 250, sonra 250. Biz bunları birleştirmek zorundayız.

MessageFramer iki aşamada çalışıyor:

Aşama 1 - Başlık: İlk 8 byte için bekliyoruz. Sihirli sayı kontrol, mesaj türü, payload boyutunu öğreniyoruz.

Aşama 2 - Payload: Artık boyutunu biliyoruz. Mesela 2000 byte payload var diyordu başlık. 200 byte geldiyse, 1800 daha bekliyoruz. Tamamduğunda, Message nesnesi oluşturup dispatcher'a veriyoruz.

Böylece 10 MB'lık bir resim bile sorun olmaz. 64 KB puffer ile bile, büyük mesajları işleyebiliriz.

13. LoginHandler

Kullanıcı giriş yaptığında çalışan handler. "Benim adım Ali, beni sisteme al!" dediğinde, LoginHandler devreye giriyor.

Yaptığı Kontroller:

Kullanıcı adı geçerli mi? 3-16 karakter, sadece harf-rakam-underscore mi? Değilse, hatamı söylüyoruz. "Adında özel karakter var, kullanma."

Başka birisi bu adı kullanıyor mu? Evet ise, "Maalesef adını başkası kullanıyor, başka seç" diyoruz.

Geçti her kontrol? OK! "Giriş başarılı, session ID = 12345 senin" diyoruz. state'i ACTIVE yapıyoruz. SessionRegistry'ye ekliyoruz. Ve tüm BAĞLI istemcilere "HEY! Ali sisteme girdi!" mesajı yayınlıyoruz. Herkes bunu alınca bilir ki Ali online'da.

14. FileHandler

Dosya işlemleri için sorumlu handler. En karmaşık handler bu. Neden? Çünkü dosya açıp kapatma, bir dosyayı birden çok kişi aynı anda görmesi, canvas senkronizasyonu, hepsi burada. Ayrıca disk operasyonları yavaş olduğu için, bunu ayrı thread'lerde yapıyor (DiskPool).

Dosya Oluştur (handleCreate):

Kullanıcı "Yeni Dosya - 800x600" dedi. FileHandler bunu DiskPool'a gönderiyor. DiskPool thread'i: beyaz 800x600 resim yaratıyor, PNG olarak diske yazıyor, metadata oluşturuyor.

Bittiğinde, istemciye "OK tamamdır, dosya ID = 5" yanıtı gidiyor. Tüm istemcilere de "Yeni dosya var!" haber veriliyor.

Dosya Aç (handleOpen):

Kullanıcı var olan bir dosyayı açıyor. Fakat burada ince bir yer var. Dosya 5 zaten açık olsa da başka birisi tarafından, canvas diske yazıldığından beri değişmiş olabilir. O kişinin en son görüntüsünü almalıyız. Yoksa diskteki eski versiyon gösterir. Yanlış olur.

Bunun için, FileHandler kontrol ediyor: dosyayı açan başka biri var mı? Varsa, ona "Hey, sen bu dosyayı açmışsın, şu an canvas durumunu gönder" diyor (CANVAS_SNAPSHOT_REQ). O kişi snapshot gönderiyor. Snapshot geliyor, yeni açan kişiye veriliyor. Eğer kimse açmamışsa, disk'ten okuyuyor.

Dosya Kaydet (handleSave):

Kullanıcı Ctrl+S yaptı. Harita veri sunucuya geliyor. FileHandler bunu DiskPool'a gönderiyor. DiskPool PNG olarak yazıyor. Bittiğinde state SAVING'den ACTIVE'e dönüyor.

Dosya Listesi (handleList):

Kullanıcı "Bana tüm dosyaları göster" dedi. FileHandler FileStore'dan tüm metadata alıyor, bunları paketliyor ve TÜMAPTA istemcilere (broadcast) gönderiyor. Herkes dosya listesini güncelliyor.

15. DrawHandler

Çizim handler. En basit handler. Neden? Çünkü sadece istediği dosyayı açan diğer istemcilere mesajı yayınlıyor.

Kullanıcı A dosya 3'de, noktadan (100,100) noktaya (200,200) mavi çizgi çiziyor. DRAW_EVENT mesajı geliyor. DrawHandler: Dosya 3, araç=LINE, renk=blue, koordinatlar, sen (A) kimin olduğunu söyle (A username'i ekle) ve bunu DRAW_BROADCAST olarak dosya 3'ü açan herkese gönder. Dosya 3'ü açan B, C, D istemcileri mesajı alıyor, kendi canvaslarında çizgi görüyorlar. Gerçek zamanlı işbirliği!

16. ClipboardHandler

Kesme-yapıştır işlemleri. Kullanıcı bir bölge seçip Ctrl+C yapıyor. ClientSession'da clipboardData kaydediliyor. Dosya 3'ü açan başkası Ctrl+V yapıyor. Pano verisi o noktaya yapıştırılıyor. Aynı dosyayı açan herkes bunu görebiliyor. Paylaşılan pano.

17. FileStore

Tüm dosyaları disk'te yöneten depo. PNG formatında kaydediyor, pixel verilerini ARGB byte dizilerine çeviriyor. Database gibi düşünebilirsiniz ama file-based.

Her dosyanın bir ID'si var: 1, 2, 3... Disk'te: file_1.png, file_2.png, file_3.png olarak kaydediliyor. Metadata de tutuluyor: dosyanın sahibi, boyutu, son değiştirilme zamanı...

Pixel format: 4 byte per pixel (ARGB). Alfa = opaklık, RGB = renk. 800x600 resim = $800 \times 600 \times 4 = 1,920,000$ byte = 1.9 MB. Bu kadar.

18. FileMetadata

Bir dosyanın tüm bilgilerini tutan hafif veri taşıyıcısı. Adı ne? Sahibi kim? Kaç byte? Ne zaman değişti?

İçinde: fileId, filename, owner, width, height, lastModified. Hepsi bu. Dosya listesi gönderirken, FileStore tüm dosyaların metadata'sını gönderir, pixel verilerini değil. Böylece çok az data transfer oluyor.

19. SessionRegistry

Sistem'in "telefonlar rehberi". Şu an kimler online? Kimlerin açık dosyaları var? Bir dosyayı kim açmış? Hepsi burada. Çok önemli bir sınıf.

Yapısı:

Harita 1 - Session ID ile arama: "Session 12345 kim?" Hemen bulur.

Harita 2 - Username ile arama: "Ali sistem'de mi?" Hemen bulur. Böylece başkası "Ali" adında giriş yapamaz (double login engellenir).

Broadcast İşlevleri:

broadcastToAll(mesaj): Tüm istemcilere "Birisi sisteme girdi" mesajı yayınla. Exclude kimliğini kullanabilir - "Ben'e gönderme, ben zaten biliyorum" gibi.

broadcastToFileViewers(fileId, mesaj): Dosya 3'ü açan tüm istemcilere mesaj gönder. Öteki dosyaları açan istemciler ilgilenmez.

Eğer RabbitMQ modunda ise, bu fonksiyonlar broker üzerinden çalışıyor (fanout exchange). Eğer TCP modunda ise, tüm istemcileri manual olarak gezdip gönderiliyor.

20. AutoSaveScheduler

Sistem "kendi kendini kaydeden" mekanizması. Her 60 saniyede bir alarm çalıyor, tüm açık dosyaları diske kaydediyor. Böylece sunucu çöpse bile, son 60 saniyenin verisi kaybı en çok olur.

Nasıl çalışıyor? SessionRegistry'den tüm açık dosya ID'lerini topluyor. Her dosya için FileStore'dan pixel verisi okuyuyor (disk'ten)... bekle, disk'ten mi? Evet, canvas her istemcide tutuluyor (backBuffer). Dosya açık istemcilerden snapshot alması gerekirse, snapshot yollanıyor, ama auto-save sadece diskteki versiyon saklanıyor. Yani bilinen versiyon.

Hata olursa, hata log'u tutuluyor ama sistem çalışmaya devam ediyor.

21. MQBroker

RabbitMQ aracısıyla iletişim kuran sunucu tarafı adapter. Bağlantı açıyor, kuyrukları ve exchange'leri deklarasyon ediyor, mesaj yayınlıyor.

Yapılandırma:

Host: rabbitmq.example.com (veya localhost). Port: 5672 (AMQP default portu).
VirtualHost, kullanıcı, şifre: rabbitmq konfigürasyonunda belirlenen.

Topoloji Kurulumu:

Paint.server kuyruğunu deklarasyon ediyor: tüm istemcilerden gelen mesajlar buraya giriyor.

Paint.global fanout exchange: USER_JOIN, USER_LEAVE, FILE_LIST_RESP, bu exchange'e gidiyor. Hepsi broadcast türü mesaj.

Paint.file.{fileId} exchange'leri lazy declaration: Dosya 5 üzerinde çizim ilk yapılırken, exchange oluşturuluyor. Dosya kapatılırsa exchange silinebiliyor (auto-delete).

Mesaj Gönderme:

sendToClient(replyTo, frame): Point-to-point, bir istemciye doğrudan mesaj.

broadcastGlobal(frame): Fanout exchange'e mesaj, herkese geliyor.

broadcastToFile(fileId, frame): Dosya exchange'ine mesaj, o dosyayı açanlar alıyor.

22. MQServerTransport

RabbitMQ modunda NIOSelector yerine kullanılan consumer. Paint.server kuyruğundan mesajları okuyor, işliyor, sonuç gönderiyor.

İşlem Akışı:

basicConsume(paint.server): Paint.server kuyruğundan mesaj almaya başla. Autoack=false, manual ACK ile.

Mesaj geldiğinde, body byte dizisi, AMQP headers'tan sessionId ve replyTo çıkart.

ClientSession bul, yoksa oluştur. MessageDispatcher'a göster. Sonuç döndür. BasicAck ile onayda.

Keepalive sweeper: Ayrı thread, 30 saniye interval'de kontrol. Eğer bir session 90 saniye ses çıkmazsa, close flag atıyor, kapatılıyor.

BÖLÜM III: İstemci Katmanı (Client)

Şimdi arkadaşlar, istemci tarafına geçiyoruz. Burada kullanıcı ile doğrudan temas kurulacak yer. Swing GUI (click, drag, menu) ve ağ iletişiminin beraber çalıştığı katman.

23. PaintClient

Tüm istemci uygulamasının başlama noktası. Main metodu burada. Ne yapıyor? Swing'in look-and-feel'ini sistem temasına ayarlıyor, EDT (Event Dispatch Thread) üzerinde MainFrame oluşturuyor, login dialog'u açıyor. O kadar basit.

24. ConnectionManager

İstemcinin "kalp ve beyni". Bağlantı yaşam döngüsünü ve ağ iletişimini yönetiyor. Çok önemli olan: TCP ve RabbitMQ modlarını kapsüllediği için, GUI katmanı hangisini kullandığını bilmez. İkiside aynı API kullanıyor.

TCP Modu Başlatma:

connect("localhost", 9090) → Socket aç, sunucuya bağlan. NetworkReader ve NetworkWriter thread'leri başlat.

RabbitMQ Modu Başlatma:

connectMQ("localhost", 5672) → RabbitMQ aracısına bağlan. Client-side özel kuyruğu oluştur, subscribe et. MQNetworkReader ve MQNetworkWriter başlat.

Sağlanan Metotlar:

sendLoginReq(username): Giriş mesajı gönder (MessageEncoder kullanarak)

sendFileCreateReq, sendFileOpenReq, sendFileSaveReq, ...: Dosya işlemleri

sendDrawEvent: Çizim olayı (araç, renk, koordinatlar)

sendClipboardCopy, sendClipboardCut, sendClipboardPasteReq: Pano işlemleri

disconnect(): Bağlantı kapat, cleanup yap

25. NetworkReader

TCP modundaki mesaj alıcı. Ayrı daemon thread'te çalışır. Blocking InputStream'den okur. Bir mesaj gelene kadar bekler.

Mesaj geldiğinde? Başlık (8 byte) okur, magic kontrol eder, payload boyutunu öğrenir. Payload (payloadLen byte) okur. Message nesnesi oluştur, SwingUtilities.invokeLater() ile EDT'de onMessage callback'i çağır. GUI thread'inde gösterebilmesi için EDT'ye koyuyor.

Bağlantı kesintisi varsa (EOFException), onDisconnect callback'i çağırıyor. GUI state'i "Disconnected"e alıyor.

26. NetworkWriter

TCP modundaki mesaj gönderici. BlockingQueue kullanarak thread-safe çalışıyor. EDT'den send(byte[]) çağrısı gelirse, veriler queue'ye konuyor. Daemon writer thread'i queue'den alıp OutputStream'e yazıyor. Block olmadığı için, GUI responsive kalıyor.

27. MQNetworkReader

RabbitMQ modundaki mesaj alıcı. BasicConsumer kullanarak client kuyruğundan mesaj tüketiyor. Login olmadan: client exclusive queue + global fanout exchange'e subscribe. LOGIN_OK alındıktan sonra: sessionId header'ı sağlanıyor. Dosya açılınca: o dosya exchange'ine subscribe (notifyFileOpened). Dosya kapatılınca: unsubscribe (notifyFileClosed).

28. MQNetworkWriter

RabbitMQ modundaki mesaj gönderici. NetworkWriter gibi çalışıyor. BlockingQueue'den frame alıp, paint.server kuyruğuna yayınlıyor. Header'lara: replyTo (client kuyruğu adı) ve sessionId (login'den sonra) ekleniyor. RabbitMQ otomatik yönlendirme yapıyor.

29. MainFrame

Ana uygulama penceresi. Bütün GUI öğelerinin toplandığı yer. Ekranın solunda dosya listesi, üstünde toolbar, ortasında tabbed pane (açık canvaslar), altında status bar. Bütün bunun koordinatörü MainFrame.

Yapısı:

ToolbarPanel: Araç seçme, renk seçme, stroke kalınlığı seçme. "Pencil mi, Line mi?" burada.

FileListPanel: Sol sidebar'da tüm dosyaları listele. "Dosya 1 (Ali), Dosya 2 (Ayşe)" şeklinde. Double-click → o dosya açılıyor, yeni tab oluşuyor.

JTabbedPane: Her açılan dosya için bir tab ve CanvasPanel. Dosya 1 tab'ında çiziyorken, Dosya 2 tab'ına geçebiliyoruz. Current toolbar ayarları (renk, araç) tablalar arasında paylaşılıyor.

Menu: Dosya (Yeni, Kaydet, Yenile), Edit (Copy, Cut, Paste), Server (Bağlan, Ayırılan).

Status Bar: "Bağlantı durumu: Çevrimiçi", "File saved", vb. mesajlar.

Mesaj İşleme:

onMessage() metodu tüm sunucu mesajlarını alıyor. Gelen mesajın türüne göre: LOGIN_OK mi? Durumu güncelle. FILE_LIST_RESP mi? FileListPanel'i güncelle. DRAW_BROADCAST mi? Aktif canvas'da çizgi göster. USER_JOIN mi? Status bar'da "Ali katıldı" yaz.

30. CanvasPanel

Gerçek çizim yüzeyimiz. 800x600 resolution. JPanel'den extends ediyor, mouse event'lerini yakalayıp, çizim işlemleri yönetiyor.

Teknik Detaylar:

Dual-buffer rendering: backBuffer (kalıcı, sunucudan alınan versiyon) ve overlayBuffer (geçici, şu anda çizilen stroke). Kullanıcı çizim yaparken, overlayBuffer'a çiziliyor (lokal feedback). Mouse release'de stroke gönderiliyor sunucuya. Sunucudan confirmation gelince (veya diğer kullanıcılardan DRAW_BROADCAST), backBuffer'a merge ediliyor. Çok realist görünüyor.

Araçlar:

- Pencil: Serbest çizim
- Line: Düz çizgi
- Rect: Dikdörtgen
- Ellipse: Elips
- Fill: Renk doldur (bucket)
- Eraser: Silgi (beyaz pixel yaz)
- Selection: Ctrl+C/X/V için seçim alanı

31. ToolbarPanel

Basit toolbar. 6 araç butonu, renk seçici, stroke kalınlığı combobox. Kullanıcı burada araç seçiyorsa, CanvasPanel'a "Şimdi Pencil'i kullan" diye haber veriliyor.

32. FileListPanel

Sol sidebar. JList kullanıyor. Sunucudan FILE_LIST_RESP gelince, liste güncelleniyor. "dosya_adı (sahibi)" formatında gösteriliyor. Double-click → FileHandler.handleOpen() çağrılıyor, dosya açılıyor.

33. LoginDialog

İlk açıldığında gösterilen modal dialog. Mode seçimi (Socket/RabbitMQ), sunucu host, port, username alanları. Mode değiştiğinde port otomatik değişiyor (9090 → 5672 veya tersi). Connect butonu clicked → ConnectionManager.connect() veya connectMQ() çağrılıyor.

Başarılı bağlantıda dialog kapanıyor, MainFrame görünüyor. Hata varsa, error popup gösteriliyor, tekrar deneyebilmesi için dialog açık kalıyor.

Sistem Genel Yapısı ve İş Akışı

Mimarinin Katmanları:

Arkadaşlar, bu sistem üç ana katmandan oluşuyor. En üstte GUI (Swing), ortada bağlantı (ConnectionManager ve Network), en altta protokol ve ağ (Wire Format + Transport).

Presentation Layer (Swing)

MainFrame, CanvasPanel, ToolbarPanel, FileListPanel, LoginDialog

Connection Abstraction (ConnectionManager)

TCP ve RabbitMQ modlarını şeffaflştırır

Transport Layer

TCP: NetworkReader/Writer + NIOSelector

RabbitMQ: MQNetworkReader/Writer + MQServerTransport

Protocol Layer

MessageEncoder/Decoder, MessageFramer

Foundation Layer

MessageType, FSMState, ProtocolConstants, Message

Gerçek Zamanlı Çizim Örneği Detaylı Akış:

Ali dosya 3'de resim yapıyor. Noktadan (100,100) (200,200) mavi çizgi çizmek istiyoruz.

1. Ali mouse'u (100,100)'de basıyor → CanvasPanel.onPressed() çağrılıyor
2. Ali (200,200)'ye sürüklüyor → overlayBuffer'a çizgi gösteriliyor (local feedback)
3. Ali mouse'u bırakıyor → onReleased() çağrılıyor
4. CanvasPanel DRAW_EVENT mesajı oluşturuyor: fileId=3, tool=LINE, color=blue, x1=100, y1=100, x2=200, y2=200
5. ConnectionManager.sendDrawEvent() çağrılıyor
6. TCP: NetworkWriter queue'ye mesajı konuyor, daemon thread gönder
7. RabbitMQ: MQNetworkWriter paint.server kuyruğuna publish ediyor
8. Sunucu mesajı alıyor → NIOSelector (TCP) veya MQServerTransport (RabbitMQ)
9. MessageDispatcher DRAW_EVENT'i DrawHandler'a yönlendirir
10. DrawHandler: "Dosya 3 açan kimler var?" diye SessionRegistry'ye sorur
11. Dosya 3'ü açan tüm istemciler bulunur (Ali, Bob, Carol olsun)
12. DRAW_BROADCAST mesajı bu üçüne gönderilir (Ali'ye de)
13. Her istemcinin NetworkReader/MQNetworkReader mesajı alıyor
14. MainFrame.onMessage() DRAW_BROADCAST işliyor
15. CanvasPanel.onDrawBroadcast() backBuffer'a çizgiyi merge ediyor
16. paint() metodu çağrılıyor, çizgi ekranda görünüyor
17. Ali, Bob ve Carol aynı anda çizgiyi görüyorlar!

Başlıca Tasarım Kararları ve Neden Önemli:

Dual-Transport Architecture: Biz bir proje yaptık ama iki moda uyuyor. Hocamız TCP ister mi, RabbitMQ ister mi, ikisinde de çalışıyor. ConnectionManager abstraction'ı sayesinde GUI hiçbir şey değiştirmeden çalışıyor.

Non-Blocking Server (NIO): Binlerce istemciyi tek thread'le yönetebiliyoruz. Eğer her istemci için thread açsaydık, server ölürdü. NIO sayesinde scalable ve hızlı.

Asenkron Disk I/O: Dosya okuma-yazma yavaş. DiskPool kullanarak, bu işler ayrı threadlerde yapılıyor. Sunucu bloklamamış oluyor. Mesajlar gelmeye devam ediyor.

FSM-based Dispatching: Her client'in durumuna göre mesajlar işleniyor. Hata handling çok temiz ve güvenilir oluyor. State transition'lar açık ve anlaşılır.

Dual-Buffer Rendering: Canvas'ta çizim responsiv gözüküyor. Local feedback (overlayBuffer) anında, server confirmation sonrası commit ediliyor. Bağlantı yavaşsa bile GUI freeze olmuyor.

RabbitMQ Fanout Exchange'ler: Doğru mesaj doğru kişilere gidiyor. Broadcast yapılırken sunucu hiç bir şey yapmak zorunda kalmıyor. RabbitMQ hepsini halletmiş, very efficient.

Auto-Save Scheduler: Periyodik olarak tüm açık dosyalar kaydediliyor. Sunucu çöpse, data loss en çok 60 saniye. Güvenilir, otomatik, user action gerekmez.

Canvas Snapshot Senkronizasyonu: Yeni birisi bir dosyaya katılırsa, o anın canvas'ı tutarlı hale getiriliyor. Disk versiyonu değil, live versiyonu kullanıyor. Böylece everyone on the same page.